Universidad Técnica Estatal de Quevedo (UTEQ)

Facultad de Ciencias de la Computación (FCC)

Carrera de Ingeniería de Software

Asignatura: Aplicaciones Distribuidos (7mo Nivel / Semestre)

TA-IND-01 — Informe Académico Individual:

Análisis Crítico de un Sistema Distribuido Real

Apache Kafka en modo KRaft: transparencias, CAP/PACELC, comunicación y tolerancia a fallos

---

Estudiante: Kenny Isaías España Vera

Paralelo: 7mo SOFTWARE A

Fecha: 13 de noviembre de 2026

Repositorio GitHub:

https://github.com/Luispaca2002/TA-IND-01-Informe-Acad-mico-Individual-An-lisis-Cr-tico-de-Sistemas-Distribuidos-Unidad-1-.git

Quevedo – Ecuador

Resumen

Este informe analiza Apache Kafka, una plataforma de streaming de eventos basada en un log particionado y replicado, en su modo KRaft, que sustituye a ZooKeeper por un quórum de controladores basado en Raft. El análisis aplica los conceptos de la Unidad 1: transparencias de distribución, teorema CAP y su extensión PACELC, modelos de comunicación y tolerancia a fallos. Se contrastaron seis artículos científicos con documentación oficial, un incidente de producción documentado y un clúster propio de tres nodos desplegado con Docker Compose. Los resultados muestran que Kafka no es CP ni AP por naturaleza: con `acks=all`, `min.insync.replicas=2` y elección no limpia deshabilitada se comporta como PC/EC, rechazando escrituras antes que perder datos confirmados y pagando latencia de replicación en operación normal. Al detener un nodo, el clúster reeligió líderes y conservó el quórum, pero el cliente que usaba ese nodo como punto de arranque falló, lo que evidencia que la transparencia de fallos termina en el borde del cliente. Frente a RabbitMQ, Kafka gana en retención y paralelismo por partición, pero pierde en enrutamiento flexible y simplicidad operativa.

Palabras clave: Apache Kafka, KRaft, teorema CAP, PACELC, tolerancia a fallos, RabbitMQ.

3. Introducción

Una cola de mensajes tradicional resuelve un problema concreto: desacoplar a quien produce un trabajo de quien lo ejecuta. El broker guarda el mensaje hasta que un consumidor lo confirma y luego lo elimina. Ese modelo funciona bien para distribuir tareas, pero se vuelve limitante cuando el mismo flujo de datos debe alimentar a varios sistemas con ritmos distintos, cuando se necesita volver a procesar eventos pasados o cuando el volumen obliga a repartir la carga entre muchas máquinas. Apache Kafka aborda ese escenario con otra abstracción: un log de solo escritura al final (*append-only*), dividido en particiones, donde cada mensaje recibe un desplazamiento (*offset*) secuencial y permanece almacenado según una política de retención, sin importar si ya fue leído [1], [2]. El consumidor, no el broker, lleva la cuenta de por dónde va.

La adopción industrial confirma que el modelo escala. Raptis y Passarella, en una revisión sistemática de unos 70 trabajos, describen a Kafka como una plataforma ampliamente usada para streaming en red, con LinkedIn como caso de referencia [1]. La propia LinkedIn reportó en 2019 más de 100 clústeres, más de 4000 brokers, más de 100 000 topics, 7 millones de particiones y más de 7 billones (*trillions*) de mensajes diarios [3]. Conviene leer esa cifra con cautela: es un dato autorreportado por la empresa que creó Kafka, no una medición auditada, y tiene varios años. Aun así, da una idea del orden de magnitud. En la literatura revisada para este informe, Kafka aparece en manufactura en la nube [4], monitoreo de presión de aire en tiempo real [5], gestión de datos industriales con OPC UA [6] y plataformas IoT monitoreadas con Elastic Stack [7].

Elegí Kafka por una razón técnica precisa: es uno de los pocos sistemas en los que los compromisos que estudia la Unidad 1 no están ocultos en el diseño, sino expuestos como parámetros. Con `acks`, `min.insync.replicas` y `unclean.leader.election.enable`, el operador decide cuánta consistencia, disponibilidad y latencia quiere en cada topic [2]. Además, el sistema acaba de completar un cambio arquitectónico de fondo: la versión 4.0, publicada el 18 de marzo de 2025, es la primera que funciona sin ZooKeeper y depende por completo del quórum KRaft [8]. Analizar Kafka hoy implica analizar ese nuevo plano de control.

El informe combina tres fuentes de evidencia: seis artículos científicos revisados y subrayados (Anexo B), documentación oficial y un incidente real de producción, y un clúster de tres nodos en modo KRaft que desplegué localmente para observar la replicación y el comportamiento ante la caída de un nodo.

4. Arquitectura del sistema

4.1. Productores

Un productor publica registros formados por una clave y un valor en un topic [1]. La clave determina la partición de destino: el particionador por defecto aplica un hash sobre la clave, de modo que todos los registros con la misma clave caen en la misma partición y conservan su orden relativo [2]. Esta decisión de diseño tiene una consecuencia directa: Kafka garantiza orden por partición, no por topic. El productor también agrupa registros en lotes y puede comprimirlos. Lu et al. compararon gzip, lz4 y snappy sobre mensajes JSON de 520 a 600 bytes y eligieron lz4 por su equilibrio entre tasa de compresión y velocidad, una elección adecuada para procesamiento

sensible a la latencia [6].

4.2. Brokers, topics y particiones

Un clúster está formado por varios brokers, y las particiones de un mismo topic se distribuyen entre ellos [5]. La partición es la unidad básica de paralelismo: es una secuencia ordenada de mensajes en la que cada nuevo registro se agrega al final y recibe un offset secuencial [1]. Cada partición se replica según un factor de replicación RF. Una réplica actúa como líder y atiende las lecturas y escrituras de esa partición; las demás son seguidoras que copian al líder y lo reemplazan si falla [1]. Con P particiones y RF réplicas, el clúster aloja $P \times RF$ réplicas en total, repartidas entre los brokers.

4.3. Segmentos de log

En disco, cada partición no es un único archivo sino una secuencia de segmentos. Solo el segmento activo recibe escrituras; cuando alcanza un tamaño o una antigüedad configurados, se cierra y se abre uno nuevo [2]. La retención trabaja sobre segmentos completos: al vencer el plazo o superar el tamaño máximo, se eliminan segmentos enteros, lo que es mucho más barato que borrar mensajes individuales. Este diseño explica por qué Kafka puede retener días de datos sin degradar las escrituras: todas las operaciones de escritura son anexos secuenciales y las lecturas por offset se resuelven ubicando el segmento cuyo offset base es el inmediato inferior.

4.4. Grupos de consumidores

Los consumidores se organizan en grupos. Dentro de un grupo, cada partición se asigna a un único consumidor, lo que reparte la carga sin que dos miembros procesen el mismo mensaje [1], [2]. De ahí sale un límite que conviene tener presente al dimensionar:

$$C_{\text{activos}} = \min(C, P), \tag{1}$$

donde C es el número de consumidores del grupo y P el número de particiones. Un cuarto consumidor sobre un topic de tres particiones queda ocioso. Los offsets confirmados por cada grupo se guardan en un topic interno, `__consumer_offsets`, administrado por un coordinador de grupo [2]. La literatura revisada contiene aquí una imprecisión: Lu et al. afirman que KRaft registra el offset de cada mensaje y que el consumidor lo busca ahí [6]. KRaft gestiona los metadatos del clúster (brokers, topics, particiones, líderes); los offsets de consumo viven en `__consumer_offsets` y la posición de cada registro es intrínseca al log de la partición [2], [9].

4.5. De ZooKeeper al consenso interno KRaft

Hasta la serie 3.x, Kafka dependía de ZooKeeper para almacenar los metadatos del clúster y coordinar la asignación de particiones y la recuperación ante fallos [5], [7]. Uno de los brokers actuaba como controlador y, cada vez que se elegía uno nuevo, debía cargar el estado desde ZooKeeper antes de poder operar [9]. Esto implicaba operar dos sistemas distribuidos distintos,

con sus propios mecanismos de seguridad, monitoreo y fallo, y añadía tiempo a cada conmutación de controlador.

KIP-500 propuso reemplazar ZooKeeper por un quórum de controladores que gestiona un log de metadatos replicado mediante Raft; el líder de ese log es el controlador activo [9]. Raft descompone el consenso en elección de líder, replicación del log y seguridad, y usa temporizadores aleatorios para resolver las elecciones [10]. Como cualquier protocolo de mayoría, un quórum de N votantes tolera

$$f = \left\lfloor \frac{N-1}{2} \right\rfloor \tag{2}$$

fallos simultáneos. Con $N = 3$ se tolera un fallo; con $N = 5$, dos. La ventaja práctica es que los controladores seguidores ya tienen los metadatos replicados en memoria, por lo que la conmutación no exige recargar el estado desde un sistema externo [9]. La versión 4.0 eliminó por completo el modo ZooKeeper [8]. Lu et al. ya desplegaron su marco industrial con tres brokers gestionados por KRaft sobre Kafka 3.7.0 [6], mientras que Zhou et al. y Pacella et al. todavía trabajaron con ZooKeeper [4], [5].

4.6. Diagrama de arquitectura

La Figura 1 resume la arquitectura analizada.

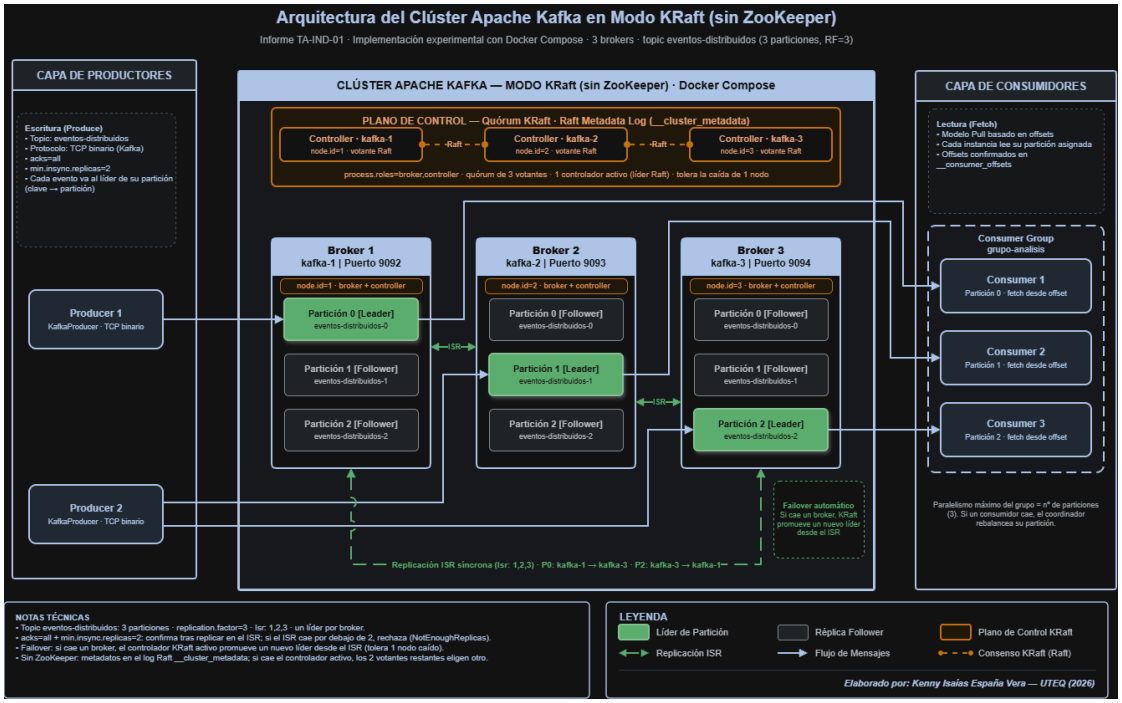


Figura 1: Arquitectura de Apache Kafka en modo KRaft: productores, quórum de controladores, brokers con particiones líder y seguidoras, y grupos de consumidores. Elaboración propia a partir del esquema del Anexo A.

El diagrama parte del bosquejo dibujado a mano del Anexo A. En esa primera versión ubiqué los componentes que reconocía en la lectura inicial: productores, un bloque de brokers y consumidores, con flechas de envío y lectura. Al contrastarlo con la documentación y con los artículos, el esquema resultó insuficiente en tres puntos. Primero, no distinguía el plano de control

del plano de datos; en la versión limpia el quórum de controladores KRaft aparece separado del flujo de mensajes, porque su función es mantener metadatos y no transportar registros. Segundo, trataba al broker como unidad de almacenamiento, cuando la unidad real es la partición; por eso el diagrama final muestra particiones con su réplica líder y sus seguidoras repartidas entre nodos. Tercero, las flechas del bosquejo sugerían que el broker empuja mensajes al consumidor; el diagrama corregido invierte esa dirección para reflejar que el consumidor solicita lotes a partir de su offset [1], [2]. Este último error es instructivo: el propio artículo de Raptis y Passarella menciona que Kafka admite consumo push y pull, aunque su ejemplo detallado describe un modelo pull [1]; la documentación oficial justifica explícitamente el diseño pull [2].

5. Análisis con los conceptos de la Unidad 1

5.1. Transparencias de distribución

Van Steen y Tanenbaum definen la transparencia de distribución como la capacidad de ocultar al usuario que procesos y recursos están repartidos en varias máquinas, y advierten que la transparencia total no siempre es deseable ni alcanzable [11]. Kafka es un buen ejemplo de esa advertencia: oculta lo que puede ocultar sin costo y expone lo que, si se ocultara, impediría al cliente tomar decisiones correctas.

Acceso. Garantizada. Todos los clientes usan el mismo protocolo binario propio sobre TCP, independientemente del broker o del sistema operativo del servidor [2], [7]. Un productor en Python y un consumidor en Java ven el mismo topic con la misma semántica.

Ubicación. Parcial. El cliente no necesita saber en qué broker reside cada partición: recibe una lista de servidores de arranque (*bootstrap servers*), pide los metadatos y a partir de ellos se conecta directamente al líder de cada partición [2]. Sin embargo, la ubicación se filtra en dos puntos. El cliente debe conocer al menos una dirección válida para arrancar, y cada broker anuncia una dirección (`advertised.listeners`) que debe ser alcanzable desde el cliente. En despliegues con Docker este segundo punto es una fuente clásica de errores: el broker anuncia un nombre interno de la red de contenedores que el host no puede resolver.

Migración. Delegada al gestor. Mover réplicas de un broker a otro es posible mediante reasignación de particiones, y los clientes se adaptan al refrescar sus metadatos [2]. Pero el sistema no decide por sí mismo cuándo migrar: el rebalanceo de carga entre brokers es una tarea del operador o de herramientas externas, y la revisión de Raptis y Passarella lo identifica como desafío abierto de investigación, junto con el particionamiento adaptativo según la carga [1].

Reubicación. Parcial, con participación del cliente. Cuando el liderazgo de una partición cambia mientras un productor está escribiendo, el broker antiguo rechaza la petición indicando que ya no es líder; el cliente refresca metadatos y reintenta [2]. El recurso se reubica en caliente, pero la transparencia depende de que el cliente tenga reintentos y tiempos de espera bien configurados. No es el servidor quien la garantiza.

Replicación. Garantizada en la vista, no en la semántica. El cliente nunca se dirige a una seguidora: escribe y lee a través del líder, y las copias son invisibles para él [1]. Pero el nivel de durabilidad que obtiene depende de un parámetro que el propio productor elige (`acks`), así que la replicación no es transparente en sus garantías: el cliente debe saber que existe para

configurarla bien.

Concurrencia. Garantizada dentro de la partición. Varios productores pueden escribir en paralelo sobre la misma partición; el líder serializa los anexos y asigna offsets en un orden total [1]. Dentro de un grupo, cada partición pertenece a un único consumidor, por lo que no hay competencia entre consumidores por el mismo registro [2]. Lo que Kafka no ofrece es orden global entre particiones: si la aplicación lo necesita, debe diseñar sus claves para que los eventos relacionados caigan en la misma partición.

Fallos. Parcial. La reelección de líder es automática: cuando el líder de una partición falla, Kafka selecciona otro entre las réplicas sincronizadas [5]. Pero el cliente sí percibe el fallo, como errores reintentables o, si la recuperación excede sus tiempos de espera, como excepciones. Mi propio experimento (Sección 6) lo mostró: el servidor reeligió líderes, pero el cliente que dependía del nodo caído falló. Kafka oculta la pérdida de datos confirmados; no oculta la interrupción.

Kafka garantiza, entonces, acceso, replicación (en la vista) y concurrencia por partición; comparte con el cliente la ubicación, la reubicación y los fallos; y delega la migración al operador. La razón es coherente con su diseño: un sistema que prioriza rendimiento y control explícito prefiere entregar al cliente la información necesaria antes que esconderla detrás de garantías que tendría que pagar con latencia.

5.2. Teorema CAP y extensión PACELC

El teorema CAP plantea que, ante una partición de red, un sistema distribuido debe elegir entre consistencia y disponibilidad. Abadi señala que CAP solo describe el comportamiento durante fallos y que, en operación normal, el compromiso que más influye en el diseño es el de latencia frente a consistencia; propone PACELC para unificar ambos: si hay partición (P), elegir entre disponibilidad (A) y consistencia (C); en otro caso (E), elegir entre latencia (L) y consistencia (C) [12].

Comportamiento ante partición (P). Considérese un topic con $RF = 3$, productor con `acks=all`, `min.insync.replicas`$= m = 2$ y `unclean.leader.election.enable=false`. Con `acks=all`, el líder solo confirma una escritura cuando todas las réplicas del conjunto ISR la tienen; si el ISR cae por debajo de m, el productor recibe `NotEnoughReplicas` y la escritura se rechaza [2]. La condición de disponibilidad de escritura de una partición es, por tanto,

$$\text{escritura aceptada} \iff |ISR| \geq m, \tag{3}$$

y el número de réplicas que pueden fallar sin bloquear escrituras es $RF - m = 1$. Si una partición de red aísla al líder de sus dos seguidoras, el líder no puede reducir el ISR por su cuenta, porque los cambios de ISR deben ser aceptados por el controlador [13]; sin quórum ni ISR suficiente, no confirma nuevas escrituras. Del lado mayoritario, el controlador elige un nuevo líder entre las réplicas que estaban sincronizadas. El resultado es que no existen dos líderes capaces de confirmar escrituras divergentes, es decir, se evita el *split-brain* sobre datos confirmados, a cambio de que el lado minoritario deje de aceptar escrituras. Es una elección PC.

El matiz importante es que esa elección no es una propiedad de Kafka, sino de esta configuración. Con `unclean.leader.election.enable=true`, una réplica fuera del ISR puede convertirse

en líder como último recurso, lo que restablece la disponibilidad pero puede perder datos ya confirmados [2]. Con $m = 1$, el líder aislado sigue aceptando escrituras que ninguna otra réplica tiene. Kafka es, entonces, un sistema cuyo punto en el eje CAP se fija por topic. El plano de control sí tiene una posición fija: el quórum KRaft se basa en Raft, que solo progresa con mayoría [9], [10]; una minoría de controladores aislada no puede elegir líder ni modificar metadatos.

Comportamiento en operación normal (E). Sin particiones, el compromiso es latencia frente a consistencia. Las seguidoras replican solicitando datos al líder [2], de modo que la latencia de confirmación con `acks=all` queda acotada por la seguidora más lenta del ISR:

$$T_{\text{acks=all}} \approx T_{\text{red}} + T_{\text{anexo}} + \max_{f \in ISR \setminus \{\ell\}} T_{\text{réplica},f}, \qquad T_{\text{acks=1}} \approx T_{\text{red}} + T_{\text{anexo}}. \tag{4}$$

La evidencia experimental de Zhou et al. es consistente con este modelo. En sus pruebas sincronizadas, el topic de una partición y una réplica obtuvo el mejor rendimiento, y los topics con tres réplicas rindieron menos por el consumo de recursos del controlador durante la replicación; los autores atribuyen además la menor tasa del productor al tiempo que este espera la confirmación del líder [5]. Kafka prioriza la consistencia por defecto: desde la serie 3.x el productor usa `acks=all` y escritura idempotente como valores predeterminados [2]. Del lado de lectura, el consumidor solo ve registros confirmados por el ISR (hasta la marca de agua alta), lo que impide leer un dato que luego desaparecería tras una conmutación. La clasificación resultante es PC/EC.

Hay un detalle que suele simplificarse. Kafka escribe en disco de forma secuencial, pero por defecto no fuerza la sincronización física (*fsync*) de cada mensaje; delega el vaciado al sistema operativo y basa su durabilidad en la replicación entre brokers [2]. La consistencia que ofrece `acks=all` significa «el dato está en la memoria o el disco de todas las réplicas del ISR», no «el dato está físicamente en el plato de un disco». Pacella et al. atribuyen la durabilidad y la preservación del orden a la arquitectura basada en log [4]; la afirmación es correcta, pero la durabilidad real proviene de la combinación log más replicación, y depende de que los fallos de las réplicas no ocurran a la vez.

5.3. Modelos de comunicación

Kafka combina dos modelos. Entre clientes y brokers rige un modelo cliente-servidor: productores y consumidores abren conexiones TCP hacia los brokers y hablan un protocolo binario propio de petición y respuesta [2], [7]. Dentro del quórum de controladores la relación es entre pares: los controladores votantes se eligen entre sí y replican el log de metadatos mediante Raft, sin un coordinador externo [9], [10]. La replicación de datos entre brokers sigue un tercer patrón, líder-seguidor, en el que las seguidoras solicitan los datos al líder.

La comunicación es asíncrona y desacoplada en dos dimensiones. En el espacio, productor y consumidor no se conocen: solo comparten el nombre del topic. En el tiempo, el mensaje persiste en el broker, de modo que el consumidor puede leerlo segundos o días después [6]. Lu et al. destacan precisamente que, al almacenarse los mensajes en el broker, productor y consumidor quedan desacoplados y se evita la pérdida de datos durante la transmisión [6].

El consumo sigue un patrón pull controlado por offsets. El consumidor solicita un lote de

mensajes a partir de su posición actual, los procesa y avanza su offset [1]. La documentación oficial justifica este diseño con dos argumentos: si el consumidor se retrasa, puede ponerse al día a su propio ritmo sin ser saturado, y el modelo permite un procesamiento por lotes agresivo, mientras que un broker que empuja debe decidir si envía de inmediato o acumula sin saber si el consumidor puede procesar [2]. El costo del pull es que un consumidor sin datos podría consultar en vano; Kafka lo mitiga con solicitudes que esperan en el broker hasta que haya datos o venza un plazo (*long polling*) [2].

El retraso de un grupo de consumidores se mide como

$$\text{lag} = \text{LEO} - \text{offset}_{\text{confirmado}}, \tag{5}$$

donde LEO es el offset final del log de la partición. Calderon et al. usaron esta métrica para supervisar una plataforma IoT real y observaron que el lag crece con el volumen de mediciones y que, cuando se añaden dispositivos de forma abrupta, aumenta con independencia de su número [7]. El lag es la expresión cuantitativa del desacoplamiento temporal: el sistema absorbe el pico como mensajes pendientes en lugar de rechazarlos.

La posición del commit define la semántica de entrega. Confirmar el offset antes de procesar produce «como máximo una vez»; confirmarlo después produce «al menos una vez», con posibles duplicados si el consumidor falla entre procesar y confirmar [2]. Raptis y Passarella recogen trabajos que modelan precisamente la probabilidad de pérdida y de duplicación según la configuración y las condiciones de red [1].

5.4. Tolerancia a fallos

Replicación basada en ISR. El líder de cada partición mantiene la lista de réplicas sincronizadas (ISR). Un mensaje se escribe primero en el líder y luego se replica a todas las réplicas del ISR; este mecanismo permite recuperar los datos cuando un broker falla [5]. Una seguidora sale del ISR si deja de alcanzar al líder durante más de `replica.lag.time.max.ms` (30 s por defecto) y vuelve a entrar cuando se pone al día [2]. El ISR es, por tanto, un quórum dinámico: no exige mayoría fija como Raft, sino que el conjunto se ajusta a las réplicas que efectivamente están al día. Esto permite que un topic con $RF = 3$ siga aceptando escrituras con una sola réplica viva si $m = 1$, algo imposible en un protocolo de mayoría estricta, pero también traslada al operador la responsabilidad de fijar m correctamente.

Detección de fallos. En modo KRaft, cada broker envía latidos (*heartbeats*) periódicos al controlador activo. Si el controlador no recibe latidos durante `broker.session.timeout.ms`, aísla al broker y elige nuevos líderes para las particiones que lideraba [2], [9]. En el lado de los consumidores, cada miembro de un grupo envía latidos al coordinador del grupo; si se excede `session.timeout.ms`, el coordinador lo da por caído y reasigna sus particiones en un rebalanceo [2]. Ambos mecanismos son detectores de fallos basados en tiempo: un temporizador corto detecta antes, pero confunde pausas largas del recolector de basura de la JVM con caídas reales.

Reelección sin pérdida de mensajes confirmados. Como un mensaje confirmado con `acks=all` ya está en todas las réplicas del ISR, cualquier miembro del ISR elegido como nuevo

líder lo contiene. Con la elección no limpia deshabilitada (valor por defecto), Kafka prefiere dejar la partición sin líder antes que promover una réplica desactualizada [2]. El número de fallos que soporta un dato confirmado sin perderse es $|ISR| - 1$. Zhou et al. confirman que, al fallar el líder de una partición, Kafka selecciona automáticamente otro líder, y advierten que el número de particiones y el factor de replicación afectan de forma significativa al rendimiento [5]. La literatura revisada por Raptis y Passarella añade alternativas complementarias, como puntos de control para la recuperación de mensajes, clústeres redundantes para recuperación ante desastres y la observación de que el tamaño de lote influye directamente en la tasa de pérdida cuando la red es inestable [1].

Incidente real documentado: PagerDuty, 28 de agosto de 2025. PagerDuty publicó el análisis de dos caídas de su clúster Kafka el mismo día [14]. Un error lógico en una funcionalidad nueva creaba un productor Kafka por cada petición a su API, en lugar de reutilizar uno compartido. Kafka mantiene metadatos por cada productor, y en el pico el clúster llegó a registrar cerca de 4,2 millones de productores adicionales por hora, unas 84 veces la tasa habitual. El uso de memoria *heap* de los brokers creció hasta que la JVM entró en un ciclo continuo de recolección de basura y agotó la memoria; el fallo empezó en un broker y se propagó al resto. Los equipos de respuesta sospecharon primero de un problema de hardware, añadieron capacidad y retiraron el broker afectado, pero otros brokers siguieron quedándose sin memoria. La primera mitigación fue duplicar el heap mediante reinicio escalonado; al ampliar el despliegue de la funcionalidad al 75 % se repitió el problema y fue necesario revertirla. Según la empresa, no se perdieron eventos ya aceptados, en parte gracias al patrón *Transactional Outbox* [14].

El incidente expone el límite del modelo de fallos que asume la replicación. El ISR protege frente a fallos independientes de tipo caída (*fail-stop*) de un nodo. Aquí la causa era una carga compartida que afectaba a todos los brokers por igual: replicar no ayuda cuando todas las réplicas reciben el mismo patrón dañino. La recuperación vino de capas externas al broker (reversión del cambio, patrón outbox, más memoria), no del mecanismo de replicación. Para mi propio experimento, con un heap limitado a 384 MB por nodo, el caso muestra que el dimensionamiento de memoria no es un detalle de laboratorio: el estado que el broker mantiene por cliente consume heap y puede derribar el clúster entero.

6. Experimentación propia

6.1. Entorno y configuración

Desplegué un clúster de tres nodos en modo KRaft con Docker Compose, con contenedores `kafka-1`, `kafka-2` y `kafka-3` basados en la imagen `apache/kafka:latest` y expuestos en los puertos 9092, 9093 y 9094. Cada nodo se limitó con `KAFKA_HEAP_OPTS=Xmx384M -Xms256M"`, de modo que el heap máximo total del clúster es $3 \times 384 = 1152$ MB. El límite responde a la memoria disponible en mi equipo y es deliberadamente bajo frente a un despliegue de producción. Sobre el clúster creé el topic `eventos-distribuidos` con 3 particiones, factor de replicación 3 y `min.insync.replicas=2`, lo que da $3 \times 3 = 9$ réplicas: cada nodo aloja una copia de cada partición.

Con tres nodos en el quórum, la Ecuación (2) da $f = 1$: el plano de control debe sobrevivir a

la caída de un nodo. Con $RF = 3$ y $m = 2$, la Ecuación (3) indica que las particiones deben seguir aceptando escrituras con un nodo menos. El experimento busca verificar ambas predicciones y observar qué percibe el cliente.

6.2. Resultados

La Figura 2 muestra los tres contenedores en ejecución. Es la condición previa del experimento: los tres nodos activos y participando en el clúster.

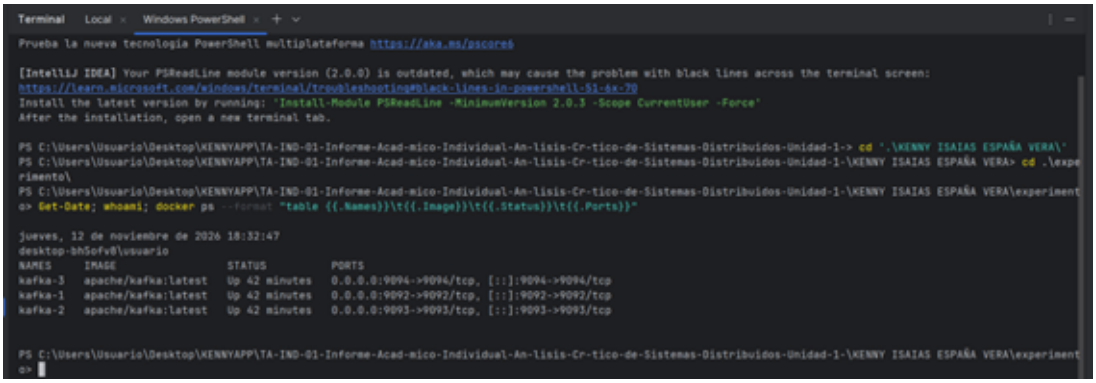


Figura 2: Contenedores `kafka-1`, `kafka-2` y `kafka-3` activos en Docker Compose.

La Figura 3 registra la creación de `eventos-distribuidos` con 3 particiones y factor de replicación 3. La operación de crear un topic es una escritura sobre el log de metadatos: el controlador activo la registra y la replica en el quórum antes de que los brokers asignen las réplicas [9]. Las advertencias `Connection to node ... (localhost/127.0.0.1:...) could not be established` que aparecen en la captura no impidieron la creación del topic, pero son un hallazgo en sí mismas: cada broker anuncia a los clientes una dirección `localhost:<puerto>`, válida desde el host pero no desde dentro de otro contenedor, donde `localhost` es el propio contenedor. Es la filtración de la transparencia de ubicación descrita en la Sección 5.1: el cliente necesita que la dirección anunciada sea alcanzable desde donde se ejecuta.

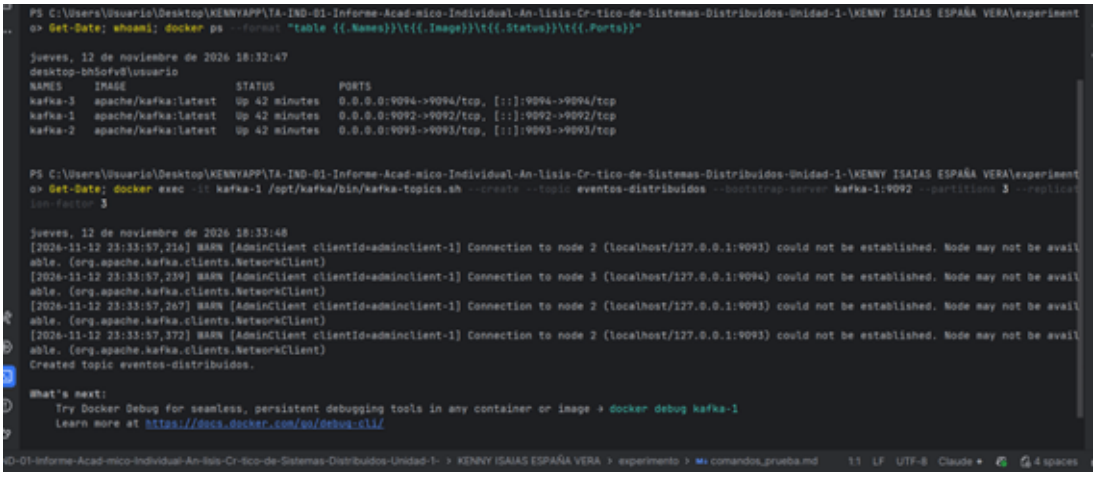


Figura 3: Creación del topic `eventos-distribuidos` (3 particiones, factor de replicación 3).

La Figura 4 muestra el estado del topic en condiciones normales. Para cada partición, el

campo `Replicas` lista los tres nodos y el campo `Isr` reporta `1,2,3`: las tres réplicas están sincronizadas con el líder. El campo `Leader` indica qué nodo atiende las lecturas y escrituras de cada partición: la partición 0 quedó liderada por el nodo 1, la 1 por el nodo 2 y la 2 por el nodo 3, y la columna `Configs` confirma `min.insync.replicas=2`. La asignación por defecto repartió el liderazgo entre los tres brokers para que ninguno concentre todas las escrituras, que es la base del escalamiento horizontal descrito en la Sección 4 [1]. Un ISR completo significa que, en este estado, un dato confirmado con `acks=all` existe en los tres nodos y sobreviviría a la pérdida de dos de ellos.

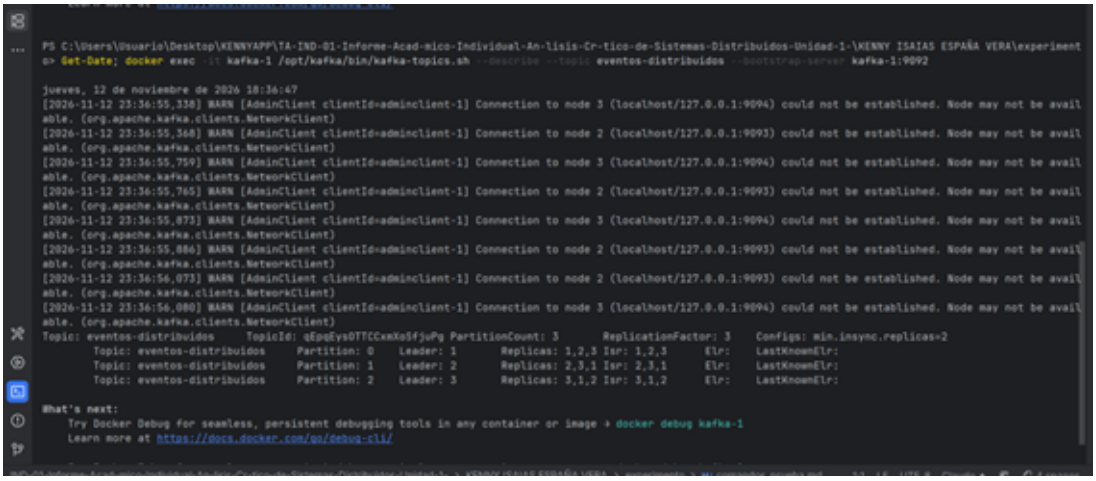


Figura 4: Estado del topic en operación normal: `Isr: 1,2,3` en todas las particiones.

A continuación detuve `kafka-1` con `docker stop` y consulté el clúster desde `kafka-2`, todavía indicando `kafka-1:9092` como servidor de arranque. La Figura 5 muestra el resultado: el cliente emitió la advertencia `Couldn't resolve server kafka-1:9092 from bootstrap.servers as DNS resolution failed` y terminó con `KafkaException: Failed to create new KafkaAdminClient`, causada por `ConfigException: No resolvable bootstrap urls given in bootstrap.servers`.

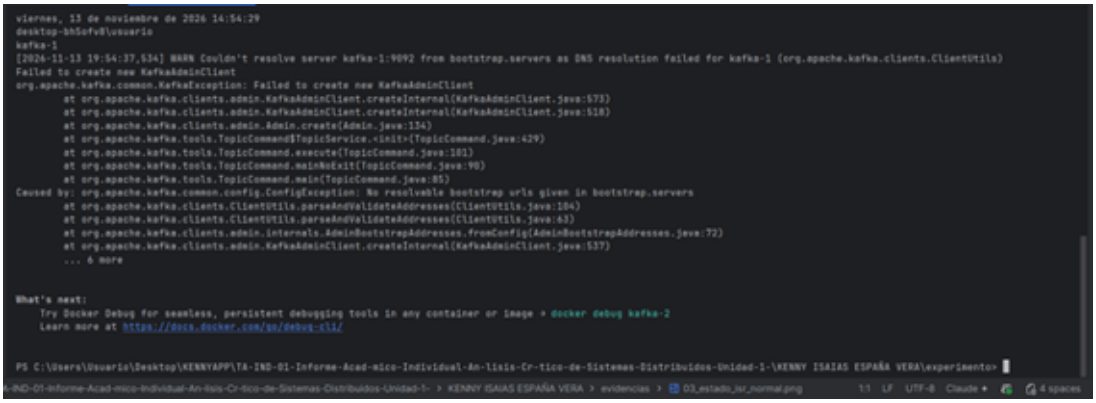


Figura 5: Fallo del cliente tras detener `kafka-1`: el único servidor de arranque indicado no se pudo resolver (`ConfigException`).

Para separar el fallo del cliente del estado real del clúster, repetí la consulta cinco segundos después de detener `kafka-1`, esta vez desde `kafka-3` y usando `kafka-3:9094` como servidor de

arranque. La Figura 6 muestra el estado del topic con un nodo caído.

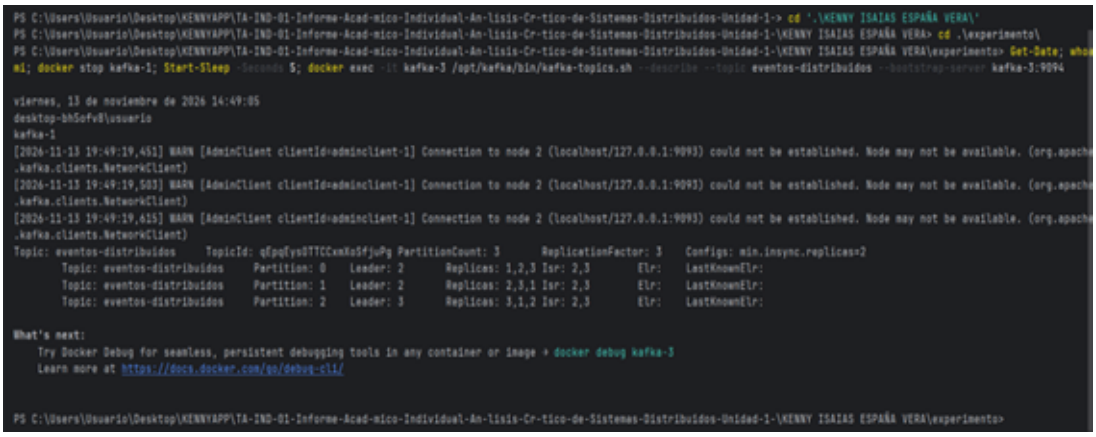


Figura 6: Estado del topic tras detener `kafka-1`: la partición 0 pasa a `Leader: 2` y todas las particiones quedan con dos réplicas sincronizadas (nodos 2 y 3).

6.3. Interpretación técnica

Lo que hizo el clúster (Figura 6). Antes de la caída, el nodo 1 lideraba la partición 0. Tras detenerlo, el controlador activo dejó de recibir sus latidos, lo aisló y eligió un nuevo líder entre las réplicas del ISR [2], [9]: la partición 0 pasó a `Leader: 2`, el siguiente nodo de su lista de réplicas asignadas que seguía sincronizado. Las particiones 1 y 2 conservaron a sus líderes (nodos 2 y 3). El ISR de las tres particiones se redujo de tres a dos miembros (`2,3`, o `3,2` en la partición 2), mientras que el campo `Replicas` siguió listando al nodo 1, porque la asignación no cambia: cuando el nodo vuelva y se ponga al día, reingresará al ISR. Todo ocurrió en menos de los cinco segundos de espera del comando, sin intervención manual.

Con $|ISR| = 2$ y `min.insync.replicas=2`, la Ecuación (3) se cumple en las tres particiones: las escrituras con `acks=all` siguen siendo aceptadas, así que el topic no perdió disponibilidad de escritura. Tampoco hay pérdida de datos confirmados: un mensaje confirmado con `acks=all` antes de la caída ya estaba en los nodos 2 y 3, y el nuevo líder de la partición 0 salió precisamente de ese conjunto [5]. En el plano de control, dos de tres votantes siguen siendo mayoría, de modo que el quórum KRaft siguió operando y pudo registrar los cambios de liderazgo [10]. Las dos predicciones planteadas al inicio de la sección se cumplieron. El margen, sin embargo, quedó en cero: una segunda caída dejaría el ISR por debajo de m y el topic rechazaría escrituras, que es el comportamiento PC analizado en la Sección 5.2. Conviene precisar también el alcance de la prueba: la ausencia de pérdida se deduce del estado del ISR y del diseño de la replicación, no de un conteo de mensajes producidos y consumidos antes y después de la caída.

Lo que vio el cliente (Figura 5). El fallo de la Figura 5 no proviene del clúster. Al detenerse el contenedor `kafka-1`, el DNS interno de Docker deja de resolver su nombre, y como ese era el único servidor de arranque indicado, el cliente no pudo contactar a ningún broker: falló al validar su configuración, antes de enviar una sola petición. No es un rechazo de escritura, que habría aparecido como `NotEnoughReplicas` [2], ni un problema de quórum, como demuestra la Figura 6, tomada con el mismo nodo caído. La corrección es de configuración del cliente:

indicar los tres brokers en `-bootstrap-server` para que cualquiera de los sobrevivientes pueda entregarle los metadatos.

Este contraste es la observación más útil del experimento. El servidor cumplió su parte de la transparencia de fallos: reeligió líderes en segundos y conservó el ISR por encima del mínimo. La transparencia se rompió en el borde del cliente, que es justamente lo que la Sección 5.1 clasificó como transparencia parcial: el clúster ocultó la caída, pero un cliente atado a un único nodo la expuso por completo.

6.4. Limitaciones del experimento

Los tres nodos comparten CPU, memoria, disco y kernel en una sola máquina, así que la independencia de fallos es ficticia: un problema del host derriba a los tres. Detener un contenedor simula una caída limpia (*fail-stop*), no una partición de red; para observar el comportamiento PC descrito en la Sección 5.2 habría que aislar un nodo con `docker network disconnect` manteniéndolo vivo. Finalmente, el experimento no mide latencias ni rendimiento, por lo que no permite validar el compromiso EL/EC con datos propios; para eso me apoyo en los resultados de Zhou et al. [5].

7. Ejemplos del entorno del estudiante

7.1. Caso 1: saturación del SGA de la UTEQ en periodos críticos

Como estudiante he observado que el Sistema de Gestión Académica (SGA) responde con lentitud, muestra errores o cierra la sesión durante los periodos de matriculación y en las horas previas al cierre de entrega de tareas. No tengo acceso a su arquitectura interna, por lo que el análisis es inferencial: los síntomas son compatibles con un sistema de petición y respuesta síncrono en el que todas las solicitudes compiten por los mismos recursos, típicamente un servidor de aplicaciones y una base de datos central.

Desde la Unidad 1, el fenómeno es una pérdida de transparencia de concurrencia: idealmente cada estudiante no debería percibir que miles de compañeros usan el sistema al mismo tiempo [11], pero la latencia y los errores le revelan exactamente eso. El cuello de botella es estructural: si el componente central no escala horizontalmente, la demanda pico se convierte en tiempos de espera y, al agotarse conexiones o memoria, en fallos por sobrecarga.

Kafka ofrecería dos mecanismos útiles, aunque no para todo. Primero, el particionamiento horizontal por clave: si las solicitudes se particionan por identificador de estudiante o de curso, la carga se reparte entre brokers y los eventos de un mismo estudiante conservan su orden [1]. Segundo, la absorción de picos: en lugar de rechazar solicitudes, el sistema las acumula como lag y los consumidores las procesan a su ritmo [7]. La propuesta tiene un límite claro. Kafka sirve para desacoplar el trabajo que no necesita respuesta inmediata (notificaciones, cálculo de promedios, verificaciones posteriores a la entrega), pero no elimina la operación síncrona central: el estudiante necesita saber que su tarea quedó registrada. Introducir una cola entre la entrega y su confirmación cambia la experiencia a «recibido, en proceso», es decir, introduce consistencia eventual en un punto donde el usuario espera una respuesta definitiva.

7.2. Caso 2: transferencias interbancarias y mensajería sin cobertura

He observado que, en la banca móvil, algunas transferencias a otros bancos aparecen como «en proceso» y el saldo del destinatario se actualiza minutos después. En WhatsApp, cuando pierdo cobertura, el mensaje queda con el ícono de reloj y se envía solo cuando recupero la conexión. Ambos casos son manifestaciones visibles de los compromisos de PACELC [12].

WhatsApp, ante la partición de red que supone quedarse sin cobertura, elige disponibilidad local: acepta el mensaje, lo almacena en el dispositivo y lo reenvía después. Es un esquema de almacenamiento y reenvío análogo al búfer del productor Kafka, que retiene registros y reintenta cuando el broker no está disponible [2]. La consistencia es eventual: durante la desconexión, emisor y receptor tienen vistas distintas de la conversación, y convergen al restablecerse el enlace.

En la transferencia interbancaria el compromiso es el opuesto en lo que importa. El sistema puede aceptar la orden de inmediato (disponibilidad para registrar la intención), pero no refleja el dinero en la cuenta destino hasta tener la confirmación de la otra institución. Prefiere mostrar «en proceso» antes que un saldo que luego tendría que revertirse. Es un comportamiento PC/EC similar al de Kafka con `acks=all`: se paga latencia visible para no exponer un estado que podría no ser definitivo. La diferencia entre ambos casos no es tecnológica sino de costo del error: un mensaje duplicado o tardío es una molestia; un saldo incorrecto es un problema contable.

8. Análisis comparativo: Apache Kafka vs. RabbitMQ

La Tabla 1 compara ambos sistemas en seis criterios de la Unidad 1.

Tabla 1: Comparación entre Apache Kafka (modo KRaft) y RabbitMQ (serie 4.x).

Criterio	**Apache Kafka**	**RabbitMQ**
Arquitectura y enrutamiento	Log particionado y replicado; el productor elige la partición por clave; sin enrutamiento por contenido en el broker [1], [2].	Broker AMQP: los mensajes llegan a *exchanges* (direct, fanout, topic, headers) que los enrutan a colas mediante *bindings* [15].
Modelo de consumo	Pull: el consumidor solicita lotes desde su offset y controla su ritmo [1], [2].	Push predominante: el broker entrega a consumidores suscritos, regulado por *prefetch*; también admite consulta bajo demanda [4], [15].
Persistencia y retención	Retención por tiempo o tamaño, independiente del consumo; relectura desde cualquier offset [2].	El mensaje se elimina de la cola tras la confirmación del consumidor; las *streams* ofrecen retención tipo log [16].
CAP / PACELC	Configurable por topic; con `acks=all`, $m = 2$ y elección no limpia deshabilitada: PC/EC [2], [12].	Las colas de quórum usan Raft: confirman con mayoría y no progresan en minoría (PC/EC) [16].
Escalabilidad y rendimiento	Escala por particiones; mayor rendimiento del productor en las pruebas revisadas [4], [17].	Mayor rendimiento del consumidor en las pruebas de Pacella et al.; escalar una cola individual es más limitado [4].
Tolerancia a fallos y replicación	ISR dinámico con reelección automática de líder; plano de control en quórum KRaft [5], [9].	Colas de quórum y streams replicadas; la réplica clásica (*mirroring*) se eliminó en la serie 4.0 [16].

Discusión crítica. Los datos empíricos disponibles no sostienen una superioridad general de Kafka, aunque así lo afirmen algunos autores. Pacella et al. midieron ambos sistemas con 100, 1000 y 10 000 mensajes. El productor de Kafka fue ligeramente más rápido (714,21 frente a 671,55 mensajes/s con 1000 mensajes), pero el consumidor de RabbitMQ superó al de Kafka en todas las cargas (por ejemplo, 2169,57 frente a 1895,80 mensajes/s con 10 000 mensajes) [4]. Aun así, la discusión del mismo artículo atribuye a Kafka capacidades superiores de alta tasa y baja latencia. La lectura que permiten sus propios datos es más acotada: Kafka destaca cuando importa el orden y la durabilidad; RabbitMQ entrega más rápido al consumidor cuando no se exige orden estricto, como reconocen los autores en otra parte del texto [4]. Además, las cargas probadas (hasta 10 000 mensajes) son pequeñas para un sistema diseñado para flujos continuos.

Maharjan et al. compararon Redis, ActiveMQ Artemis, RabbitMQ y Kafka y concluyeron que Kafka ofrece el mayor rendimiento con un margen significativo [17]. Sus experimentos, sin embargo, se ejecutaron en una sola máquina con un productor y un consumidor y no consideraron la persistencia, que es precisamente el factor que diferencia a los dos sistemas en durabilidad. Zhou et al., comparando Kafka con ActiveMQ, reportaron retardos medios de Kafka entre 15,6 y 336,1 ms frente a 48,9 y 4374,5 ms de ActiveMQ al crecer el número de dispositivos, pero también reconocieron que ActiveMQ fue más estable gracias a su mecanismo de limitación de calidad de servicio [5].

El argumento clásico de que Kafka es «consistente» y RabbitMQ «no» quedó desactualizado. Desde que RabbitMQ eliminó la réplica clásica y adoptó colas de quórum basadas en Raft como opción replicada por defecto [16], ambos sistemas resuelven la replicación con quórum. La

diferencia de fondo hoy no está en CAP sino en el modelo de datos: Kafka es un log que se relee; RabbitMQ es un enrutador de mensajes que se consumen. Elegir entre ambos depende de si el problema exige reprocesamiento y paralelismo por partición, o enrutamiento flexible y entrega por mensaje.

9. Postura crítica

Fortalezas reales. Kafka combina tres propiedades que pocos sistemas ofrecen juntas: retención independiente del consumo, paralelismo explícito por partición y garantías de durabilidad configurables. La primera permite agregar consumidores nuevos que procesen la historia completa sin tocar al productor. La segunda convierte el escalamiento en una decisión de diseño medible ($C_{\text{activos}} = \text{mín}(C, P)$). La tercera permite ajustar cada topic a su costo de error. Los artículos revisados muestran además resultados sólidos en entornos industriales: Lu et al. reportan que el 99,9 % de los envíos se completó en menos de 1 s, un rendimiento medio de 51,96 MB/s y un recorrido completo desde la captura hasta el almacenamiento en menos de 7 ms en su banco de pruebas [6].

Limitaciones operativas. La flexibilidad tiene un precio: la configuración por defecto no es necesariamente la segura para cada caso, y los parámetros interactúan entre sí. Elegir el número de particiones es una decisión difícil de revertir, porque cambiarla altera la asignación de claves a particiones y rompe el orden por clave. La operación exige monitoreo continuo: Calderon et al. tuvieron que construir un marco completo con agentes Beats y Kibana para observar el lag, la memoria y la CPU de su plataforma [7]. El incidente de PagerDuty muestra que la memoria de la JVM y el número de clientes son variables de riesgo que no aparecen en la descripción conceptual del sistema [14]. Por último, gran parte de la literatura evalúa Kafka en máquinas virtuales o laboratorios: Lu et al. reconocen que su banco de pruebas no refleja la congestión ni la heterogeneidad de un despliegue industrial real [6].

Compromisos de diseño. Kafka cambia simplicidad del broker por complejidad del cliente: el broker no rastrea qué mensaje leyó cada consumidor, y por eso escala, pero la gestión de offsets, reintentos y duplicados recae en la aplicación. Cambia también latencia por consistencia, como formaliza PACELC [12], y orden global por paralelismo, porque solo hay orden dentro de cada partición.

Escenarios donde no se debe usar Kafka.

- **Sistemas de bajo volumen.** Un servicio que procesa unos pocos mensajes por segundo no justifica operar un quórum de controladores, varios brokers y su monitoreo. Una cola simple o incluso una tabla en la base de datos resuelven el problema con menos piezas que pueden fallar.
- **Enrutamiento complejo por contenido o cabeceras.** Si los mensajes deben distribuirse según patrones de clave o atributos de cabecera, los *exchanges* topic y headers de RabbitMQ lo resuelven en el broker [15]. En Kafka habría que crear topics adicionales o filtrar en cada consumidor.

- **Latencia sub-milisegundo y determinista sin persistencia.** Kafka añade, como mínimo, un viaje de red hasta el líder, el anexo al log y, con `acks=all`, la replicación. Los retardos medidos por Zhou et al. están en el orden de decenas a cientos de milisegundos [5]. Las aplicaciones que necesitan latencia determinista, como el control industrial coordinado, requieren mecanismos en la capa de red como Time-Sensitive Networking, donde la planificación de paquetes garantiza tiempos de envío acotados [18]. El propio trabajo de Lu et al. reconoce que el desempeño de su esquema en escenarios de muy baja latencia no fue verificado [6].
- **Colas de trabajo con confirmación y reintento por mensaje.** Cuando cada tarea debe confirmarse, reintentarse o descartarse individualmente, el modelo de offset por partición es incómodo, porque un mensaje problemático bloquea a los siguientes. Kafka 4.0 introdujo en acceso temprano un modelo de colas para este caso [8], lo que confirma que era una carencia del diseño original.

10. Conclusiones

1. **La posición de Kafka en CAP/PACELC es una decisión de configuración, no una propiedad del sistema.** Con $RF = 3$, `acks=all`, `min.insync.replicas=2` y elección no limpia deshabilitada, Kafka se comporta como PC/EC: rechaza escrituras cuando $|ISR| < m$ y paga en operación normal la latencia de la réplica más lenta del ISR. Basta cambiar m a 1 o habilitar la elección no limpia para desplazarlo hacia AP con riesgo de pérdida de datos confirmados.

2. **KRaft simplifica el plano de control sin cambiar el modelo de replicación de datos.** El clúster experimental de tres nodos conserva el quórum con un nodo caído ($f = 1$), y las particiones con $RF = 3$ y $m = 2$ siguen aceptando escrituras. El plano de control usa mayoría estricta (Raft); el plano de datos usa un ISR dinámico. Confundir ambos mecanismos, como hace uno de los artículos revisados al atribuir a KRaft el registro de offsets, lleva a diagnósticos errados.

3. **La transparencia de fallos de Kafka termina en el cliente.** Al detener `kafka-1`, el clúster reeligió en segundos al nodo 2 como líder de la partición 0 y mantuvo `Isr: 2,3`, pero el cliente que tenía a `kafka-1` como único servidor de arranque falló sin llegar a contactar al clúster. El resultado indica que la configuración del cliente (lista de arranque, tiempos de espera, reintentos, idempotencia) forma parte del diseño de tolerancia a fallos y no puede tratarse como un detalle de implementación.

4. **La replicación protege contra fallos independientes, no contra fallos correlacionados.** Tanto el incidente de PagerDuty como las limitaciones de mi propio despliegue, con tres nodos en un mismo host y heap de 384 MB, muestran que la redundancia pierde valor cuando todas las réplicas comparten la causa del fallo. Frente a RabbitMQ, que hoy también replica con Raft, la ventaja de Kafka no está en la consistencia sino en su modelo de log retenido y particionado; cuando el problema no requiere reprocesamiento ni paralelismo por partición, esa ventaja no compensa su costo operativo.

11. Declaración de uso de inteligencia artificial

En la elaboración de este informe utilicé dos asistentes de inteligencia artificial: Claude (Anthropic) y Gemini (Google). Los usé para estructurar el código LaTeX del documento, validar la sintaxis del archivo Docker Compose y depurar comandos del clúster. Claude también generó el borrador de la redacción de las secciones del informe y apoyó la búsqueda de fuentes complementarias y la extracción de metadatos bibliográficos; revisé ese borrador, contrasté sus afirmaciones con los artículos citados y lo ajusté a mi experiencia y resultados.

Realicé personalmente, sin asistencia de IA, el diseño del experimento, la ejecución del clúster de tres nodos y la captura de evidencias, el esquema de arquitectura dibujado a mano (Anexo A), la lectura y el subrayado de los artículos científicos (Anexo B) y la verificación manual de cada DOI en el sitio de su editorial. Asumo la responsabilidad por el contenido final del informe.

12. Referencias

[1] T. P. Raptis y A. Passarella, «A Survey on Networked Data Streaming With Apache Kafka,» *IEEE Access*, vol. 11, págs. 85 333-85 350, 2023. DOI: 10.1109/ACCESS.2023.3303810

[2] The Apache Software Foundation. «Apache Kafka Documentation,» visitado 13 de nov. de 2026. dirección: https://kafka.apache.org/documentation/

[3] J. Lee y W. Wu. «How LinkedIn Customizes Apache Kafka for 7 Trillion Messages per Day,» LinkedIn Engineering Blog, visitado 13 de nov. de 2026. dirección: https://engineering.linkedin.com/blog/2019/apache-kafka-trillion-messages

[4] M. Pacella, A. Papa, G. Papadia y E. Fedeli, «A Scalable Framework for Sensor Data Ingestion and Real-Time Processing in Cloud Manufacturing,» *Algorithms*, vol. 18, n.º 1, 2025, **artno** 22. DOI: 10.3390/a18010022

[5] Z. Zhou, L. Zhou y Z. Chen, «A Distributed Real-Time Monitoring Scheme for Air Pressure Stream Data Based on Kafka,» *Applied Sciences*, vol. 14, n.º 12, 2024, **artno** 4967. DOI: 10.3390/app14124967

[6] D. Lu, K. Wang, Y. Wang e Y. Shen, «The Proposal and Validation of a Distributed Real-Time Data Management Framework Based on Edge Computing with OPC Unified Architecture and Kafka,» *Applied Sciences*, vol. 15, n.º 12, 2025, **artno** 6862. DOI: 10.3390/app15126862

[7] G. Calderon, G. del Campo, E. Saavedra y A. Santamaría, «Monitoring Framework for the Performance Evaluation of an IoT Platform with Elasticsearch and Apache Kafka,» *Information Systems Frontiers*, vol. 26, págs. 2373-2389, 2024. DOI: 10.1007/s10796-023-10409-2

[8] The Apache Software Foundation. «Apache Kafka 4.0.0 Release Announcement,» visitado 13 de nov. de 2026. dirección: https://kafka.apache.org/blog/2025/03/18/apache-kafka-4.0.0-release-announcement/

[9] C. McCabe. «KIP-500: Replace ZooKeeper with a Self-Managed Metadata Quorum,» Apache Kafka Improvement Proposals, visitado 13 de nov. de 2026. dirección: https://cwiki.apache.org/confluence/display/KAFKA/KIP-500%3A+Replace+ZooKeeper+with+a+Self-Managed+Metadata+Quorum

[10] D. Ongaro y J. Ousterhout, «In Search of an Understandable Consensus Algorithm,» en *Proceedings of the 2014 USENIX Annual Technical Conference (USENIX ATC 14)*, Philadelphia, PA, USA: USENIX Association, 2014, págs. 305-319, ISBN: 978-1-931971-10-2. dirección: https://www.usenix.org/conference/atc14/technical-sessions/presentation/ongaro

[11] M. van Steen y A. S. Tanenbaum, *Distributed Systems*, 4.ª ed. distributed-systems.net, 2023, ISBN: 978-90-815406-3-6. dirección: https://www.distributed-systems.net

[12] D. J. Abadi, «Consistency Tradeoffs in Modern Distributed Database System Design: CAP is Only Part of the Story,» *Computer*, vol. 45, n.º 2, págs. 37-42, 2012. DOI: 10.1109/MC.2012.33

[13] The Apache Software Foundation. «KIP-497: Add Inter-Broker API to Alter ISR,» Apache Kafka Improvement Proposals, visitado 13 de nov. de 2026. dirección: https://cwiki.apache.org/confluence/display/KAFKA/KIP-497%3A+Add+inter-broker+API+to+alter+ISR

[14] PagerDuty. «August 28 Kafka Outages – What Happened and How We're Improving,» PagerDuty Engineering Blog, visitado 13 de nov. de 2026. dirección: https://www.pagerduty.com/eng/august-28-kafka-outages-what-happened-and-how-were-improving/

[15] RabbitMQ Team. «AMQP 0-9-1 Model Explained,» RabbitMQ Documentation, visitado 13 de nov. de 2026. dirección: https://www.rabbitmq.com/tutorials/amqp-concepts

[16] RabbitMQ Team. «Quorum Queues,» RabbitMQ Documentation, visitado 13 de nov. de 2026. dirección: https://www.rabbitmq.com/docs/quorum-queues

[17] R. Maharjan, M. S. H. Chy, M. A. Arju y T. Cerny, «Benchmarking Message Queues,» *Telecom*, vol. 4, n.º 2, págs. 298-312, 2023. DOI: 10.3390/telecom4020018

[18] Q. Lv, X. Jiang y X. Yang, «Making programmable packet scheduling time-sensitive with a FIFO queue,» *Journal of Cloud Computing*, vol. 12, 2023, **artno** 141. DOI: 10.1186/s13677-023-00518-3

13. Anexos

Anexo A: Esquema inicial de arquitectura dibujado a mano

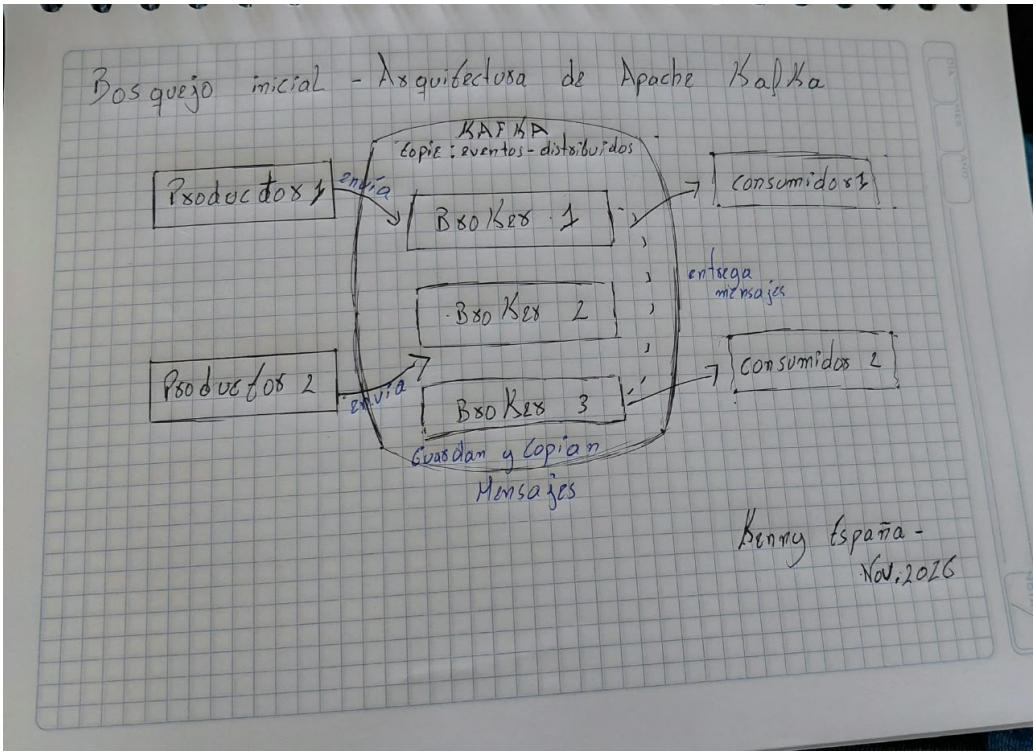
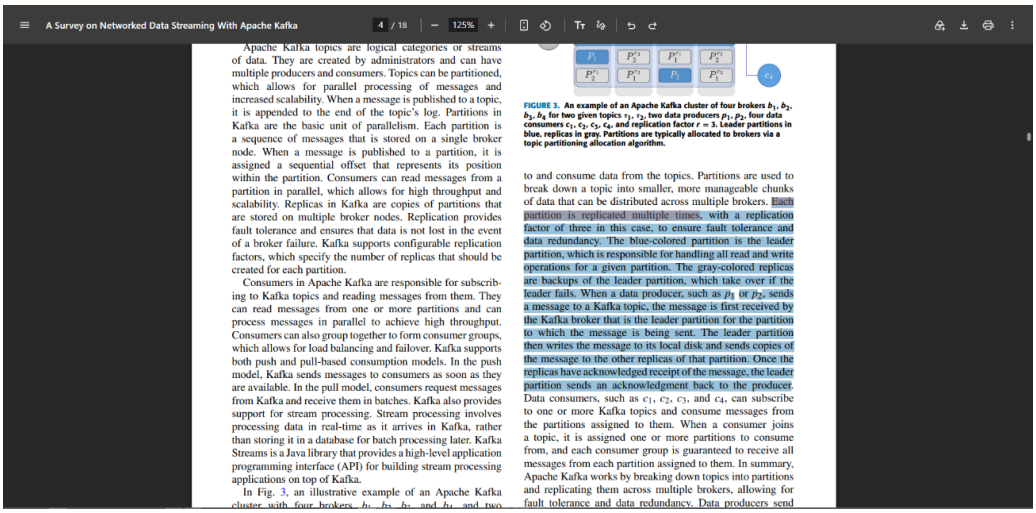


Figura 7: Bosquejo simplificado de la arquitectura de Apache Kafka, elaborado a mano con la visión general del sistema. Su evolución hacia la Figura 1 se explica en la Sección 4.6.

Anexo B: Evidencias de lectura y subrayado de fuentes científicas

Cada captura corresponde a un fragmento subrayado en el PDF original y citado en el cuerpo del informe.



Apache Kafka topics are logical categories or streams of data. They are created by administrators and can have multiple producers and consumers. Topics can be partitioned, which allows for parallel processing of messages and increased scalability. When a message is published to a topic, it is appended to the end of the topic's log. Partitions in Kafka are the basic unit of parallelism. Each partition is a sequence of messages that is stored on a single broker node. When a message is published to a partition, it is assigned a sequential offset that represents its position within the partition. Consumers can read messages from a partition in parallel, which allows for high throughput and scalability. Replicas in Kafka are copies of partitions that are stored on multiple broker nodes. Replication provides fault tolerance and ensures that data is not lost in the event of a broker failure. Kafka supports configurable replication factors, which specify the number of replicas that should be created for each partition.

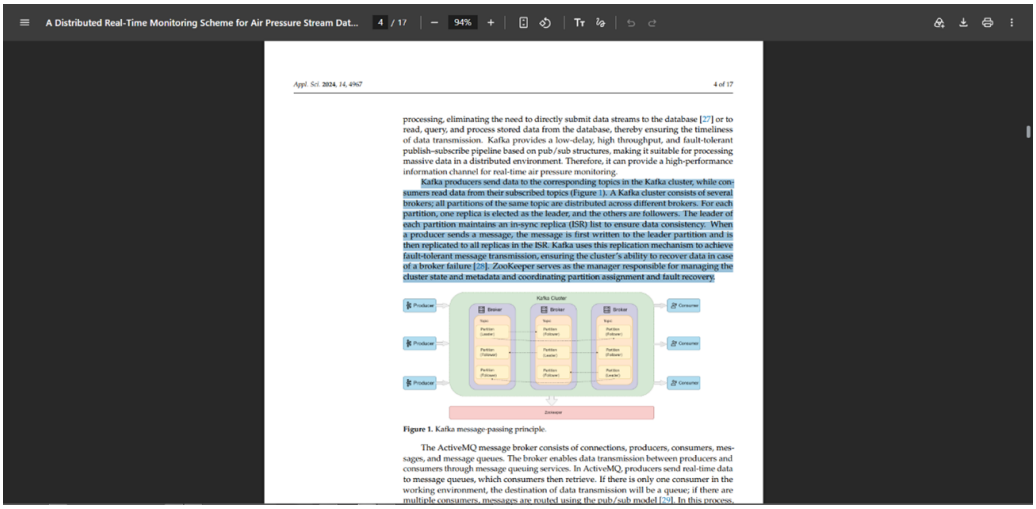
Consumers in Apache Kafka are responsible for subscribing to Kafka topics and reading messages from them. They can read messages from one or more partitions and can process messages in parallel to achieve high throughput. Consumers can also group together to form consumer groups, which allows for load balancing and failover. Kafka supports both push and pull-based consumption models. In the push model, Kafka sends messages to consumers as soon as they are available. In the pull model, consumers request messages from Kafka and receive them in batches. Kafka also provides support for stream processing. Stream processing involves processing data in real-time as it arrives in Kafka, rather than storing it in a database for batch processing later. Kafka Streams is a Java library that provides a high-level application programming interface (API) for building stream processing applications on top of Kafka.

In Fig. 3, an illustrative example of an Apache Kafka cluster with four brokers b_1, b_2, b_3, and b_4, and two

FIGURE 3. An example of an Apache Kafka cluster of four brokers b_1, b_2, b_3, b_4 for two given topics τ_1, τ_2, two data producers p_1, p_2, four data consumers c_1, c_2, c_3, c_4, and replication factor $r = 3$. Leader partitions in blue, replicas in gray. Partitions are typically allocated to brokers via a topic partitioning allocation algorithm.

to and consume data from the topics. Partitions are used to break down a topic into smaller, more manageable chunks of data that can be distributed across multiple brokers. Each partition is replicated multiple times, with a replication factor of three in this case, to ensure fault tolerance and data redundancy. The blue-colored partition is the leader partition, which is responsible for handling all read and write operations for a given partition. The gray-colored replicas are backups of the leader partition, which take over if the leader fails. When a data producer, such as p_1 or p_2, sends a message to a Kafka topic, the message is first received by the Kafka broker that is the leader partition for the partition to which the message is being sent. The leader partition then writes the message to its local disk and sends copies of the message to the other replicas of that partition. Once the replicas have acknowledged receipt of the message, the leader partition sends an acknowledgment back to the producer. Data consumers, such as c_1, c_2, c_3, and c_4, can subscribe to one or more Kafka topics and consume messages from the partitions assigned to them. When a consumer joins a topic, it is assigned one or more partitions to consume from, and each consumer group is guaranteed to receive all messages from each partition assigned to them. In summary, Apache Kafka works by breaking down topics into partitions and replicating them across multiple brokers, allowing for fault tolerance and data redundancy. Data producers send

Figura 8: Raptis y Passarella [1], p. 85336 (página 4 del PDF): replicación con líder y réplicas, y confirmación al productor tras el acuse de las réplicas. Citado en las Secciones 4 y 5.2.



Appl. Sci. 2024, 14, 4967 4 of 17

processing, eliminating the need to directly submit data streams to the database [27] or to read, query, and process stored data from the database, thereby ensuring the timeliness of data transmission. Kafka provides a low-delay, high throughput, and fault-tolerant publish–subscribe pipeline based on pub/sub structures, making it suitable for processing massive data in a distributed environment. Therefore, it can provide a high-performance information channel for real-time air pressure monitoring.

Kafka producers send data to the corresponding topics in the Kafka cluster, while consumers read data from their subscribed topics (Figure 1). A Kafka cluster consists of several brokers; all partitions of the same topic are distributed across different brokers. For each partition, one replica is elected as the leader, and the others are followers. The leader of each partition maintains an in-sync replica (ISR) list to ensure data consistency. When a producer sends a message, the message is first written to the leader partition and is then replicated to all replicas in the ISR. Kafka uses this replication mechanism to achieve fault-tolerant message transmission, ensuring the cluster's ability to recover data in case of a broker failure [28]. ZooKeeper serves as the manager responsible for managing the cluster state and metadata and coordinating partition assignment and fault recovery.

Figure 1. Kafka message-passing principle.

The ActiveMQ message broker consists of connections, producers, consumers, messages, and message queues. The broker enables data transmission between producers and consumers through message queuing services. In ActiveMQ, producers send real-time data to message queues, which consumers then retrieve. If there is only one consumer in the working environment, the destination of data transmission will be a queue; if there are multiple consumers, messages are routed using the pub/sub model [29]. In this process,

Figura 9: Zhou et al. [5], p. 4: líder y seguidores por partición, lista ISR y recuperación ante la caída de un broker. Citado en la Sección 5.4.

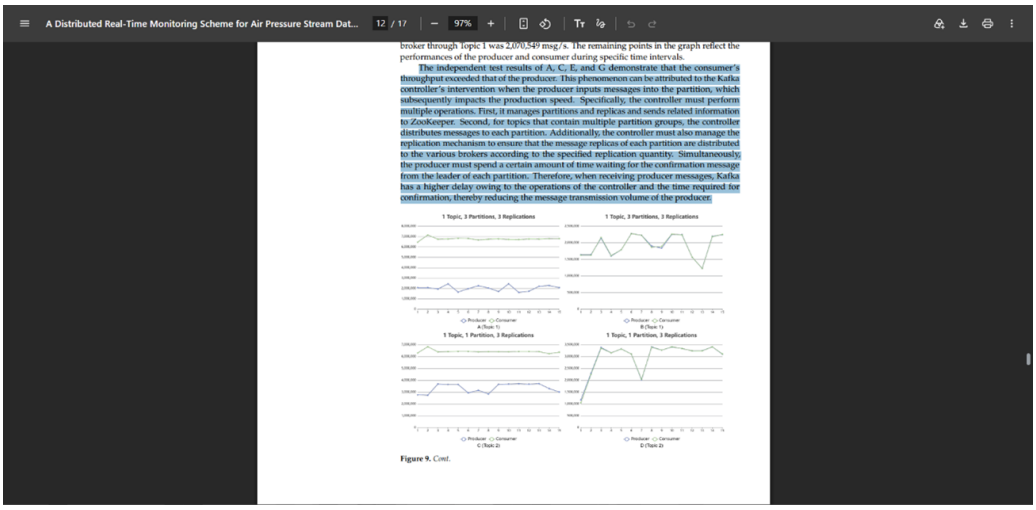


Figura 10: Zhou et al. [5], p. 12: efecto de las operaciones de replicación y de la espera de confirmaciones sobre el rendimiento del productor. Citado en la Sección 5.2 (PACELC).

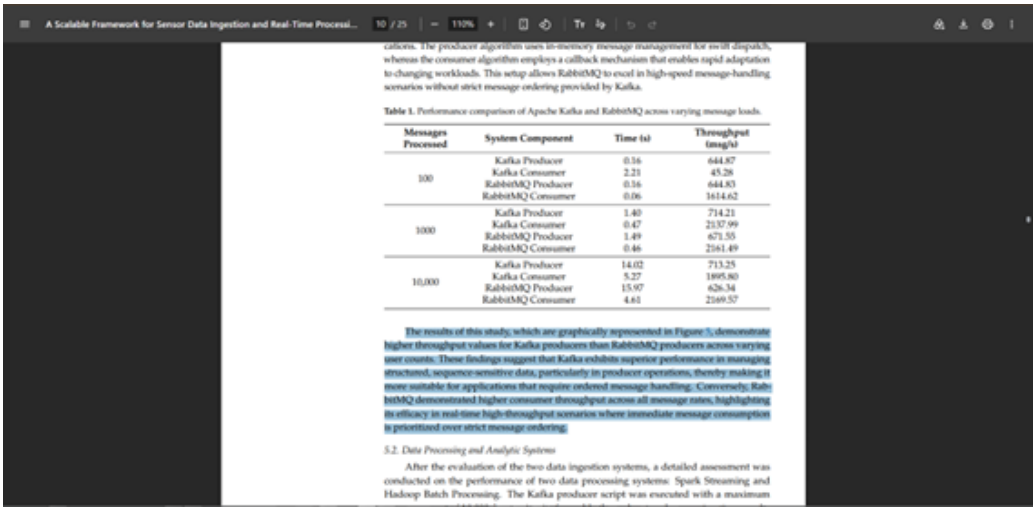


Figura 11: Pacella et al. [4], p. 10: Tabla 1 y comparación del rendimiento de productores y consumidores de Kafka y RabbitMQ. Citado en la Sección 8.